

DSA0502 – Query Processing for Data Science	
Course Faculty	Dr. B .SHYAMALA GOWRI
Reg. No.	192424050
Student Name	THARUN.V
Date	17/08/2026

CO4 AT1 – SCHEMA ANALYSIS AND DATA VOICE

Question 5 — Retail Customer Behavior Analysis

1. Data Importing & Dataset Description

Four relational datasets are imported and profiled before any analysis begins. Each file is loaded into a Pandas DataFrame (or an equivalent SQL table) and checked for shape, data types, missing values, and duplicate records using `df.info()`, `df.describe()`, `df.isnull().sum()`, and `df.duplicated().sum()`.

1.1 Customer Dataset

Contains demographic and account-level attributes for each customer.

Column	Description	Data Type
CustomerID	Unique identifier for each customer	Integer (Primary Key)
CustomerName	Full name of the customer	String
Gender	Male / Female / Other	Categorical
Age	Age of the customer in years	Integer
City / Region	Home city or region of the customer	Categorical
SignupDate	Date the customer registered	Date
CustomerSegment	Pre-assigned tier (if available), e.g. Regular/Premium	Categorical

1.2 Product Dataset

Describes the catalog of items sold.

Column	Description	Data Type
ProductID	Unique identifier for each product	Integer (Primary Key)
ProductName	Name of the product	String
Category	Product category (Electronics, Apparel, Grocery, etc.)	Categorical
SubCategory	Finer classification within a category	Categorical

Column	Description	Data Type
UnitPrice	Standard selling price per unit	Float
Brand	Manufacturer or brand name	Categorical

1.3 Sales (Transactions) Dataset

The fact table linking customers and products through individual transactions.

Column	Description	Data Type
TransactionID	Unique identifier for each sale	Integer (Primary Key)
CustomerID	Foreign key to Customer dataset	Integer (FK)
ProductID	Foreign key to Product dataset	Integer (FK)
StoreID	Foreign key to Store Location dataset	Integer (FK)
Quantity	Units purchased in the transaction	Integer
TotalAmount	Quantity × Unit Price (after discount)	Float
Discount	Discount applied, if any	Float
TransactionDate	Date and time of purchase	DateTime
PaymentMode	Cash / Card / UPI / Wallet	Categorical

1.4 Store Location Dataset

Provides geographic and operational context for each store.

Column	Description	Data Type
StoreID	Unique identifier for each store	Integer (Primary Key)
StoreName	Name/branch identifier	String
City	City where the store operates	Categorical
State / Region	Broader geographic grouping	Categorical
StoreType	Flagship / Standard / Kiosk / Online	Categorical
OpeningDate	Date the store began operations	Date

Import code pattern (Python/Pandas):

- `customers = pd.read_csv('customer.csv')`
- `products = pd.read_csv('product.csv')`
- `sales = pd.read_csv('sales.csv', parse_dates=['TransactionDate'])`
- `stores = pd.read_csv('store_location.csv')`

Figure/Note: All four files are validated for consistent key formats (e.g. CustomerID as int in both Customer and Sales tables) before joining.

2. Data Exploration

Exploratory checks establish data quality and initial intuition about the business before deeper analysis:

1. Structural check: row/column counts, data types, and memory footprint for each of the four tables.
2. Missing value audit: percentage of nulls per column; Age, Discount, and PaymentMode typically show the highest missingness in retail data and are treated with median imputation or an 'Unknown' category.
3. Duplicate check: repeated TransactionID or CustomerID rows are flagged and removed.
4. Range/sanity checks: negative Quantity, zero UnitPrice, or TransactionDate outside the business's operating window are flagged for correction.
5. Univariate exploration: distribution of Age, Gender split, spread of TotalAmount, and frequency of Category using histograms, boxplots, and bar charts.
6. Cardinality check: number of unique customers, products, and stores, used later to judge the granularity of aggregation.

2.1 Sample Summary Statistics (illustrative)

Metric	Value
Total Transactions	48,512
Unique Customers	6,340
Unique Products	1,120
Unique Stores	42
Average Transaction Value	₹1,842
Date Range	Jan 2023 – Jun 2026

Note: Values above are illustrative placeholders demonstrating the expected structure of the summary; replace with actual computed statistics once the real datasets are loaded.

3. Joining Multiple Datasets

The Sales table is the central fact table; it is joined with the three dimension tables (Customer, Product, Store Location) using left joins to preserve every transaction while enriching it with descriptive attributes.

3.1 Join Strategy

Join	Key	Type	Purpose
Sales ⋈ Customer	CustomerID	Left Join	Attach demographic & segment info to each sale
Sales ⋈ Product	ProductID	Left Join	Attach category, brand, price info to each sale
Sales ⋈ Store Location	StoreID	Left Join	Attach city/region/store type to each sale

Python implementation:

- `df = sales.merge(customers, on='CustomerID', how='left')`
- `.merge(products, on='ProductID', how='left')`
- `.merge(stores, on='StoreID', how='left')`

After joining, the resulting master DataFrame (df) contains one row per transaction enriched with customer demographics, product category/price, and store location — enabling every subsequent grouping, correlation, and

segmentation step to be performed on a single unified table. Row count is validated to equal the original Sales row count, confirming no fan-out from the joins.

Equivalent SQL: SELECT s., c.*, p.*, st.* FROM Sales s LEFT JOIN Customer c ON s.CustomerID = c.CustomerID LEFT JOIN Product p ON s.ProductID = p.ProductID LEFT JOIN StoreLocation st ON s.StoreID = st.StoreID;*

4. Grouping & Aggregation

Aggregation converts the transaction-level master table into business-relevant summaries.

4.1 Revenue by Product Category

- `df.groupby('Category')['TotalAmount'].sum().sort_values(ascending=False)`

4.2 Revenue and Footfall by Store / City

- `df.groupby(['City', 'StoreName']).agg(Revenue=('TotalAmount', 'sum'), Transactions=('TransactionID', 'count'))`

4.3 Customer-Level Aggregation (foundation for segmentation)

- `cust_agg = df.groupby('CustomerID').agg(TotalSpend=('TotalAmount', 'sum'), OrderCount=('TransactionID', 'nunique'), AvgOrderValue=('TotalAmount', 'mean'), LastPurchase=('TransactionDate', 'max'))`

4.4 Monthly Sales Trend

- `df.groupby(df['TransactionDate'].dt.to_period('M'))['TotalAmount'].sum()`

Sample aggregated output (illustrative):

Category	Total Revenue	% of Total	Avg Basket Size
Electronics	₹1.82 Cr	34%	₹3,120
Apparel	₹1.05 Cr	20%	₹1,480
Grocery	₹0.98 Cr	18%	₹640
Home & Living	₹0.76 Cr	14%	₹2,050
Beauty & Personal Care	₹0.75 Cr	14%	₹980

5. Data Segmentation

Customers, products, and stores are segmented to move from raw aggregates to actionable groups.

5.1 Customer Segmentation — RFM Analysis

Recency, Frequency, and Monetary (RFM) scoring is the primary technique used to identify high-value customers:

- Recency: days since the customer's last transaction (lower = better).
- Frequency: number of distinct orders placed (higher = better).
- Monetary: total spend across all orders (higher = better).

Each metric is split into quintiles (1–5) using `pd.qcut`, combined into an RFM score, and mapped to named segments:

Segment	Typical RFM Profile	Business Meaning
Champions	High F, High M, Low Recency	Most valuable, most loyal customers
Loyal Customers	High F, Moderate M	Consistent repeat buyers
At Risk	High past M, rising Recency	Previously valuable, going inactive
New Customers	Low F, Low Recency	Recently acquired, needs nurturing
Hibernating / Lost	Low F, High Recency	Long inactive, low engagement

5.2 Product Segmentation

- ABC Analysis: rank products by revenue contribution; 'A' items $\approx$ top 20% of SKUs driving $\approx$ 80% of revenue (Pareto principle).
- Category/Sub-category performance tiers: Fast-moving, Medium-moving, Slow-moving based on quantity sold per period.

5.3 Store / Regional Segmentation

- High-performing vs underperforming stores based on revenue per store and revenue per square foot (if available).
- Urban vs Tier-2/Tier-3 city performance comparison.

Segments feed directly into Section 10 recommendations (targeted retention offers for 'At Risk', loyalty rewards for 'Champions', reactivation campaigns for 'Hibernating').

6. Correlation Analysis

Correlation analysis quantifies relationships between numeric variables in the joined dataset (Age, Quantity, UnitPrice, Discount, TotalAmount, OrderCount, TotalSpend) using Pearson's correlation coefficient and a heatmap.

- `corr_matrix = df[['Age','Quantity','UnitPrice','Discount','TotalAmount']].corr()`
- `sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')`

6.1 Expected / Illustrative Relationships

Variable Pair	Typical Direction	Business Interpretation
Quantity $\leftrightarrow$ TotalAmount	Strong Positive	More units purchased drives higher basket value
Discount $\leftrightarrow$ Quantity	Moderate Positive	Discounting stimulates bulk purchase behavior
UnitPrice $\leftrightarrow$ Quantity	Negative	Higher-priced items are bought in smaller quantities
OrderCount $\leftrightarrow$ TotalSpend	Strong Positive	Frequent buyers are naturally higher-value customers
Age $\leftrightarrow$ TotalAmount	Weak / Near Zero	Age alone is a poor predictor of spend; segment matters more

A statistically weak correlation between Age and spend suggests marketing should target behavioral segments (RFM) rather than demographic age bands alone.

7. Outlier Detection

Outliers are detected on TotalAmount, Quantity, and customer-level TotalSpend using two complementary methods:

7.1 IQR (Interquartile Range) Method

- `Q1, Q3 = df['TotalAmount'].quantile([0.25, 0.75])`
- `IQR = Q3 - Q1; lower = Q1 - 1.5*IQR; upper = Q3 + 1.5*IQR`
- `outliers = df[(df['TotalAmount'] < lower) | (df['TotalAmount'] > upper)]`

7.2 Z-Score Method

- `z = (df['TotalAmount'] - df['TotalAmount'].mean()) / df['TotalAmount'].std()`
- `outliers = df[z.abs() > 3]`

7.3 Interpreting Outliers

- Legitimate high-value transactions: bulk/corporate orders — retain, and consider a separate B2B segment.
- Data entry errors: negative amounts, Quantity = 0 with non-zero TotalAmount — correct or drop.
- Fraud/refund anomalies: unusually high discount + high quantity combinations — flag for audit.

Boxplots per Category are used to visually confirm outliers and check whether extreme values cluster in specific product categories (commonly Electronics and Home & Living due to high unit prices).

8. Time-Based Analysis

TransactionDate is decomposed into Year, Month, Quarter, Day-of-week, and Hour to reveal temporal demand patterns.

- `df['Month'] = df['TransactionDate'].dt.month_name()`
- `df['Quarter'] = df['TransactionDate'].dt.quarter`
- `df['DayOfWeek'] = df['TransactionDate'].dt.day_name()`

8.1 Key Temporal Views

- Monthly revenue trend line — identifies seasonality (e.g., festive-season spikes in Oct–Nov, back-to-school in Jun).
- Year-over-year growth rate — $(\text{Current Year Revenue} - \text{Prior Year Revenue}) / \text{Prior Year Revenue}$.
- Day-of-week heatmap — typically shows weekend peaks for in-store footfall and weekday peaks for online/wallet transactions.
- Cohort analysis — group customers by SignupDate month and track retention/repeat-purchase rate over subsequent months.

Period	Revenue Trend (Illustrative)	Likely Driver
Q1 (Jan–Mar)	Moderate, post-holiday dip	Post-festive spending pullback
Q2 (Apr–Jun)	Steady growth	Summer/season-change wardrobe & appliance demand
Q3 (Jul–Sep)	Slight softening	Monsoon-related footfall dip in physical stores

Period	Revenue Trend (Illustrative)	Likely Driver
Q4 (Oct–Dec)	Peak	Festive season and year-end promotions

9. Visualization

The following chart types are recommended for the retail dataset, each mapped to the analytical question it answers:

Chart Type	Variables	Business Question Answered
Bar Chart	Revenue by Category	Which product categories drive the most revenue?
Line Chart	Monthly Revenue Trend	How does demand change over time?
Heatmap	Correlation Matrix / Day×Hour sales	Which variables move together? When is footfall highest?
Boxplot	TotalAmount by Category	Where are the outliers and spread differences?
Pie / Donut Chart	Customer Segment Share	What proportion of customers falls into each RFM segment?
Choropleth / Bar-by-City Map	Revenue by City/State	Which regions are top vs. underperforming?
Scatter Plot	Frequency vs Monetary	How do purchase frequency and spend relate for segmentation?
Pareto Chart	Cumulative % Revenue by Product	Which 20% of products drive 80% of revenue?

These visuals are typically built with Matplotlib/Seaborn in Python or as an interactive Power BI / Tableau dashboard with slicers for Category, City, and Date Range so business users can self-serve the analysis.

10. Key Findings, Conclusion & Recommendations

10.1 Key Findings

1. A small share of customers (the 'Champions' RFM segment) is expected to contribute a disproportionately large share of revenue, consistent with the Pareto principle.
2. Electronics and Home & Living categories show the highest average basket value but also the widest spread of outlier transactions.
3. Revenue is seasonal, peaking around festive quarters (Q4), suggesting inventory and staffing should scale accordingly.
4. Discounting shows a positive relationship with quantity purchased, but its effect on overall margin should be validated against cost data before scaling promotions.
5. A subset of stores/cities consistently outperform others; underperforming locations warrant a root-cause review (footfall, assortment, or local competition).
6. Age is a weak predictor of spend — behavioral segmentation (RFM) is more actionable than demographic-only targeting.

10.2 Business Recommendations

- Launch a loyalty/rewards program targeted at 'Champions' and 'Loyal Customers' to protect and grow this high-margin base.
- Design win-back campaigns (personalized offers, reminder emails) specifically for the 'At Risk' and 'Hibernating' segments before they churn completely.
- Apply ABC analysis results to optimize inventory — prioritize stock availability for 'A' category fast-movers and consider markdown/clearance for slow-movers.
- Investigate underperforming stores for operational or assortment issues; consider reallocating inventory or marketing spend to high-performing regions.
- Time major promotional campaigns ahead of the Q4 seasonal peak identified in the time-based analysis, and pre-position inventory accordingly.
- Audit high-outlier transactions periodically as a lightweight fraud/error-detection control.

10.3 Conclusion

The retail dataset demonstrates a classic pattern where a minority of customers and products drive the majority of revenue. Combining RFM segmentation, category-level ABC analysis, and time-based seasonality gives the business a prioritized, data-driven roadmap: retain high-value customers, optimize the product mix, and time promotions around demonstrated seasonal peaks.

PART B

Question 6 — Weather and Climate Analysis

1. Data Importing & Dataset Description

Five datasets are imported and profiled: Temperature, Rainfall, Humidity, Wind Speed, and Location. Each is read into a Pandas DataFrame with date columns parsed as datetime and checked for shape, types, and missing values.

Dataset	Key Columns	Grain
Temperature	LocationID, Date, MinTemp, MaxTemp, AvgTemp (°C)	Daily, per location
Rainfall	LocationID, Date, RainfallMM, RainyDayFlag	Daily, per location
Humidity	LocationID, Date, HumidityMorning, HumidityEvening (%)	Daily, per location
Wind Speed	LocationID, Date, WindSpeedKMH, WindDirection	Daily, per location
Location	LocationID, StationName, City, State, Latitude, Longitude, Elevation	Master/reference

Import pattern:

- `temp = pd.read_csv('temperature.csv', parse_dates=['Date'])`
- `rain = pd.read_csv('rainfall.csv', parse_dates=['Date'])`
- `humidity = pd.read_csv('humidity.csv', parse_dates=['Date'])`
- `wind = pd.read_csv('wind_speed.csv', parse_dates=['Date'])`
- `location = pd.read_csv('location.csv')`

All four measurement files share the same (LocationID, Date) grain, which is the key used for joining them into a single analytical table.

2. Data Exploration

Initial exploration establishes data completeness and sensor reliability before analysis:

1. Structural check: date range coverage per location — weather stations often have gaps from sensor downtime.
2. Missing value audit: WindSpeed and Humidity typically have the highest missingness due to sensor faults; handled via forward-fill within a station or interpolation, not simple mean imputation (weather is autocorrelated in time).
3. Range/sanity checks: MinTemp > MaxTemp, negative RainfallMM, Humidity outside 0–100%, or WindSpeed outside 0–200 km/h are flagged as sensor errors.
4. Duplicate check: duplicate (LocationID, Date) pairs across any of the four measurement files.
5. Univariate exploration: distribution of AvgTemp, RainfallMM, Humidity, and WindSpeed via histograms and boxplots, split by location.

2.1 Sample Summary Statistics (illustrative)

Metric	Value
Locations Covered	18 weather stations
Date Range	Jan 2016 – Jul 2026 (10.5 years)
Average Daily Temp (overall)	27.4 °C
Average Annual Rainfall	1,120 mm
Average Humidity	68%
Average Wind Speed	11.2 km/h

Note: Values above are illustrative placeholders demonstrating expected structure; replace with actual computed statistics once real datasets are loaded.

3. Joining Multiple Datasets

Temperature, Rainfall, Humidity, and Wind Speed are merged into one daily-level 'weather master' table using (LocationID, Date) as the composite join key, then enriched with the Location master for geographic context.

Join	Key	Type	Purpose
Temperature ∞ Rainfall	LocationID + Date	Outer Join	Combine two core daily measures without losing days present in only one file
(+) ∞ Humidity	LocationID + Date	Outer Join	Add humidity readings
(+) ∞ Wind Speed	LocationID + Date	Outer Join	Add wind speed readings
(+) ∞ Location	LocationID	Left Join	Attach City/State/Lat-Long/Elevation

Python implementation:

- `weather = temp.merge(rain, on=['LocationID','Date'], how='outer')`
- `.merge(humidity, on=['LocationID','Date'], how='outer')`
- `.merge(wind, on=['LocationID','Date'], how='outer')`
- `.merge(location, on='LocationID', how='left')`

Outer joins are preferred over left joins at the measurement-merge stage because sensor files do not always share identical date coverage; using outer joins avoids silently dropping valid readings, and resulting nulls are handled explicitly during cleaning rather than being hidden by a restrictive join.

*Equivalent SQL: SELECT * FROM Temperature t FULL OUTER JOIN Rainfall r ON t.LocationID = r.LocationID AND t.Date = r.Date ... LEFT JOIN Location l ON t.LocationID = l.LocationID;*

4. Grouping & Aggregation

The daily weather master table is aggregated to monthly, seasonal, and location levels to reveal climate patterns.

4.1 Monthly Aggregation

- `monthly = weather.groupby([weather['Date'].dt.to_period('M'), 'City']).agg(AvgTemp=('AvgTemp','mean'), TotalRainfall=('RainfallMM','sum'), AvgHumidity=('HumidityMorning','mean'), AvgWind=('WindSpeedKMH','mean'))`

4.2 Seasonal Aggregation

- `weather['Season'] = weather['Date'].dt.month.map(month_to_season) # Winter/Summer/Monsoon/Post-Monsoon`
- `weather.groupby('Season').agg(AvgTemp=('AvgTemp','mean'), TotalRainfall=('RainfallMM','sum'))`

4.3 Location-Level Aggregation

- `weather.groupby('City').agg(AvgTemp=('AvgTemp','mean'), MaxTempRecorded=('MaxTemp','max'), TotalRainfall=('RainfallMM','sum'))`

Sample aggregated output (illustrative):

Season	Avg Temp (°C)	Total Rainfall (mm)	Avg Humidity (%)
Winter (Dec–Feb)	18.6	42	58
Summer (Mar–May)	33.2	18	41
Monsoon (Jun–Sep)	27.8	812	84
Post-Monsoon (Oct–Nov)	26.1	168	70

5. Data Segmentation

Locations and time periods are segmented to move from raw readings to actionable climate categories.

5.1 Climate Zone Segmentation (by Location)

- Hot & Dry: high AvgTemp, low RainfallMM, low Humidity.
- Hot & Humid: high AvgTemp, moderate-high RainfallMM, high Humidity.
- Temperate: moderate AvgTemp, moderate rainfall, moderate humidity.
- High Rainfall / Monsoon-Dominant: RainfallMM well above the network average, concentrated in Jun–Sep.

5.2 Day-Type Segmentation

- Extreme Heat Days: MaxTemp above the location's 95th percentile.
- Heavy Rainfall Days: RainfallMM above 64.5 mm/day (standard 'heavy rain' threshold) or location-specific 90th percentile.
- High Wind / Storm-Risk Days: WindSpeedKMH above 40 km/h.
- Comfortable Days: AvgTemp between 20–28°C and Humidity between 40–60%.

5.3 Yearly Trend Segmentation

- Years are grouped and compared against the multi-year average to flag years as Above Normal, Normal, or Below Normal rainfall/temperature years — commonly done relative to a 30-year climatological baseline where available.

These segments directly support Section 8 (time-based analysis) and Section 10 (extreme-weather-driven recommendations).

6. Correlation Analysis

Correlation analysis quantifies relationships between the core climate variables — AvgTemp, RainfallMM, Humidity, and WindSpeedKMH — using Pearson's correlation coefficient and a heatmap.

- `corr_matrix = weather[['AvgTemp','RainfallMM','HumidityMorning','WindSpeedKMH']].corr()`
- `sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')`

6.1 Expected / Illustrative Relationships

Variable Pair	Typical Direction	Climatological Interpretation
Humidity ↔ Rainfall	Strong Positive	Moisture-laden air precedes and accompanies rain events
Temperature ↔ Humidity	Negative	Hot, dry-season air holds relatively lower relative humidity
Rainfall ↔ Wind Speed	Weak-Moderate Positive	Storm systems often bring both rain and gusty wind together
Temperature ↔ Rainfall	Negative (seasonal)	Monsoon months are cooler and wetter than peak summer
Elevation ↔ Temperature	Negative	Higher-elevation stations record cooler average temperatures

These relationships are validated per-location as well as pooled, since correlation strength between temperature and rainfall can vary meaningfully between a coastal city and an inland/arid one.

7. Outlier Detection

Outliers are identified on temperature, rainfall, and wind speed to distinguish genuine extreme-weather events from sensor faults.

7.1 IQR Method (per location, per season)

- `Q1, Q3 = weather.groupby(['City','Season'])['MaxTemp'].transform(lambda s: s.quantile(0.25)), ...quantile(0.75)`
- Applying IQR within (City, Season) groups avoids flagging a normal 40°C summer day in a hot region as an outlier just because it looks extreme network-wide.

7.2 Z-Score Method

- `z = (weather['RainfallMM'] - weather.groupby('City')['RainfallMM'].transform('mean')) / weather.groupby('City')['RainfallMM'].transform('std')`
- `extreme_rain_days = weather[z.abs() > 3]`

7.3 Interpreting Outliers

- Genuine extreme events: record heatwaves, cloudburst-level rainfall, cyclonic wind speeds — retained and flagged as an 'Extreme Weather Events' log for the department.
- Sensor faults: implausible spikes (e.g., 0% humidity, WindSpeed > 200 km/h with no corroborating rainfall/pressure signal) — investigated and corrected or excluded from trend analysis.
- Cross-validation: a suspected extreme value is checked against neighboring stations on the same date; a value unsupported by nearby stations is more likely a sensor error than a real event.

Boxplots per City and per Season are used to visually confirm which locations and months are most prone to genuine climate extremes (e.g., monsoon-month rainfall outliers in coastal cities).

8. Time-Based Analysis

Date is decomposed into Year, Month, Season, and Day to reveal seasonal cycles and long-term climate trends.

- `weather['Year'] = weather['Date'].dt.year`
- `weather['Month'] = weather['Date'].dt.month_name()`

8.1 Key Temporal Views

- Monthly climatology — average temperature/rainfall per calendar month across all years, showing the typical annual cycle.
- Year-over-year trend line — annual average temperature and total rainfall by year, used to detect warming trends or shifting monsoon patterns.
- Rolling averages — 7-day and 30-day moving averages smooth daily noise to reveal underlying trend direction.
- Anomaly analysis — each year's average compared to the multi-year (climatological) baseline to flag unusually hot, cold, wet, or dry years.

Period	Pattern (Illustrative)	Interpretation
Dec–Feb (Winter)	Lowest temperatures, minimal rainfall	Stable, dry, cool season
Mar–May (Summer)	Rising, peak temperatures	Pre-monsoon heat buildup
Jun–Sep (Monsoon)	Sharp rainfall spike, moderated temperature	Dominant rainy season, majority of annual rainfall
Oct–Nov (Post-Monsoon)	Gradually cooling, tapering rainfall	Transition back toward winter
Multi-Year Trend	Gradual year-on-year AvgTemp increase (illustrative)	Consistent with broader regional warming patterns; to be confirmed against actual computed trend line

9. Visualization

The following chart types are recommended for the weather dataset, each mapped to the analytical question it answers:

Chart Type	Variables	Business/Scientific Question Answered
Line Chart	Monthly/Yearly AvgTemp & Rainfall	How do temperature and rainfall trend over time?
Heatmap	Correlation Matrix / Month×Year Rainfall grid	Which variables move together? Which months/years were wettest?
Boxplot	Temperature or Rainfall by Season/City	Where is the seasonal spread widest, and where are the outliers?
Bar Chart	Total Rainfall by City	Which locations receive the most/least rainfall?
Wind Rose	WindSpeed × WindDirection	What are the dominant wind directions and speeds at a location?
Choropleth Map	Avg Temp / Rainfall by Location (Lat-Long)	How does climate vary geographically across the region?

Chart Type	Variables	Business/Scientific Question Answered
Scatter Plot	Humidity vs Rainfall	How strongly do humidity and rainfall move together?
Area Chart	Cumulative Rainfall over the Year	How does rainfall accumulate across the monsoon season?

These are typically built with Matplotlib/Seaborn/Plotly in Python or as an interactive Power BI dashboard with slicers for Location, Year, and Season, allowing meteorologists to drill into any station or period.

10. Key Findings, Conclusion & Recommendations

10.1 Key Findings

1. Rainfall and humidity are strongly positively correlated, confirming that moisture content in the air is a leading indicator of rainfall events.
2. Temperature and humidity move inversely, consistent with typical dry-hot vs humid-cool seasonal behavior.
3. The monsoon period (Jun–Sep) concentrates the large majority of annual rainfall, making water-resource planning highly sensitive to this narrow window.
4. Extreme heat and heavy rainfall days cluster in specific locations and months rather than occurring uniformly, indicating localized risk hotspots.
5. Year-over-year analysis suggests a gradual warming trend and/or shifting rainfall distribution (to be confirmed with the actual computed trend line on real data) — a pattern consistent with broader regional climate change signals.
6. A subset of extreme readings are attributable to sensor faults rather than genuine weather events, underscoring the need for cross-station validation in the data pipeline.

10.2 Recommendations

- Prioritize sensor maintenance and calibration at stations showing frequent implausible outlier readings to improve data reliability.
- Issue early-warning advisories keyed to the identified 'Extreme Heat Days' and 'Heavy Rainfall Days' thresholds for the specific locations/months where they cluster.
- Use the monsoon-concentration finding to inform reservoir and water-resource management, ensuring storage capacity aligns with the narrow high-rainfall window.
- Track the year-over-year temperature/rainfall anomaly series on an ongoing basis as a lightweight climate-trend monitor, flagging years that deviate significantly from the climatological baseline.
- Deploy the correlation findings (humidity as a rainfall precursor) to improve short-term rainfall forecasting models.
- Expand station density in regions currently showing large gaps in date coverage to improve the granularity of regional climate assessments.

10.3 Conclusion

The weather dataset reveals a clear, physically consistent seasonal cycle — humid, rain-heavy monsoons; hot, dry summers; and cool, dry winters — with strong statistical relationships between humidity and rainfall and an inverse relationship between temperature and humidity. Extreme-value analysis distinguishes genuine climate extremes from sensor error, and time-based trend analysis provides an early lens on longer-term climate shifts.

Together, these findings support both operational uses (early warnings, resource planning) and strategic uses (long-term climate trend monitoring) for the weather department.